Übung 2: Asymptotische Analyse

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie lernen, die Laufzeit und den Speicherbedarf einfacher Algorithmen in Abhängigkeit von der Eingabegröße n zu modellieren und asymptotisch mit der O-, Ω- und Θ-Notation zu beschreiben.

1. Geben Sie für jede Funktion eine möglichst enge asymptotische obere Schranke in der O-Notation an. Ordnen Sie die Funktionen anschließend nach ihrem asymptotischen Wachstum, beginnend mit der am langsamsten wachsenden Funktion.

   1. $\log_4(n) + \log_2(n)$
   2. $\frac{n}{2} + 4$
   3. $2^n + 3$
   4. $750,000,000$
   5. $8n + 4n^2$

2. Bestimmen Sie die asymptotische Laufzeit des folgenden Algorithmus in Abhängigkeit von der Anzahl n der Listenelemente. Wie verändert sich die Laufzeit, wenn anstelle einer `ArrayList` eine `LinkedList` verwendet wird? Begründen Sie Ihre Antwort.

```
public void mystery2(ArrayList<String> list) {
  for (int i = 0; i < list.size(); i++) {
    for (int j = 0; j < list.size(); j++) {
      System.out.println(list.get(i));
    }
  }
}
```

3. Modellieren Sie für jeden der folgenden Codeblöcke eine mathematische Funktion, die die Laufzeit des Codes in Abhängigkeit von n modelliert. Geben Sie dann die O-, Ω- und Θ-Notation für Ihre modellierte Laufzeitfunktion an.

   (a)
```
int x = 0;
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n * n / 3; j++) {
    x += j;
  }
}
```

   (b)
```
int x = 0;
for (int i = n; i >= 0; i -= 1) {
  if (i % 3 == 0) {
    break;
  } else {
    x += n;
  }
}
```

(c)
```
int x = 0;
for (int i = 0; i < n; i++) {
  if (i % 5 == 0) {
    for (int j = 0; j < n; j++) {
      if (i == j) {
        x += i * j;
      }
    }
  }
}
```

(d)
```
int x = 0;
if (n % 2 == 0) {
  for (int i = 0; i < n * n * n * n; i++) {
    x++;
  }
} else {
  for (int i = 0; i < n * n * n; i++) {
    x++;
  }
}
```

4. Modellieren Sie für jeden der folgenden Codeblöcke den zusätzlichen Speicherbedarf in Abhängigkeit von n durch eine mathematische Funktion. Geben Sie anschließend die O-Notation für den Speicherbedarf an.

(a)
```
List<Integer> list = new LinkedList<Integer>();
for (int i = 0; i < n * n; i++) {
  list.add(i);
}
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
  System.out.println(it.next());
}
```

(b)
```
int[] arr = {0, 0, 0};
n = arr.length;
for (int i = 0; i < n; i++) {
  arr[0]++;
}
```

5. Bestimmen Sie die asymptotische Laufzeit des folgenden Algorithmus für den Best Case und den Worst Case. Geben Sie für beide Fälle geeignete O-, Ω- und Θ-Schranken an.

```
int findPosOfFirstPrimeNumber(int[] arr) {
  for (int i = 0; i < arr.length; i++) {
    if (isPrime(arr[i]))
      return i;
  }
  return -1;
}
```

6. Geben Sie die Laufzeit der folgenden Funktion `print` im Best Case und im Worst Case jeweils in Θ-Notation in Abhängigkeit von n, der Länge des Eingabearrays `input`, an. Welche Eigenschaft hat das Eingabearray im Best Case bzw. im Worst Case? Eine Begründung der Θ-Schranken ist nicht erforderlich.

```
void print(int[] input) {
  int i = 0;
  while (i < input.length - 1) {
    if (input[i] > input[i + 1]) {
      int j = input.length;
      while (j > 2) {
        System.out.println(``uh I do not think this is sorted. please help'');
        j /= 2;
      }
    } else {
      System.out.println(``input[i] <= input[i + 1] is true'');
    }
    i++;
  }
}
```

7. Was ist die asymptotische Laufzeit der folgenden Funktion `f` in der Θ-Notation? Begründen Sie Ihre Antwort kurz.

```
void f(int n) {
  int i = 1;
  int j;
  while(i < n*n*n*n) {
    j = n;
    while (j > 2) {
      j /= 2;
    }
    i += n*n;
  }
}
```

8. Bestimmen Sie die asymptotische Laufzeit der Funktion `g` in der Θ-Notation. Begründen Sie Ihre Antwort kurz.

```
void g(int n) {
  for (int i = 1; i <= n; i *= 2) {
    for (int j = 1; j <= n; j++) {
      // Führe eine konstante Anzahl von Operationen aus
    }
  }
}
```

9. Geben Sie die Laufzeit für die folgende Funktion `f` in der Θ-Notation an. Es sollte ersichtlich sein wie Sie auf diese Laufzeit gekommen sind (bspw. sehr kurze Begründung).

```
def f(n):
  y = -1000
  while (n >= 2):
    if n % 3 == 1:
      y = y + 5
    n = n/2

  return y
```

10. Geben Sie für jede der folgenden vier Funktionen die vereinfachte O-Notation in der zweiten Spalte der Tabelle an und beantworten Sie dann die entsprechende "Wahr oder Falsch" Frage zu dieser Funktion durch Einkreisen von "W" bzw. "F".

Funktion	**Vereinfachte O-Notation**	**Wahr (W) oder Falsch (F)? (Einkreisen)**
$a(n) = 42\log_2(n)$		$a(n)$ ist in $O(n)$ W oder F
$b(n) = 4n + 3000$		$b(n)$ ist in $\Omega(n \log n)$ W oder F
$c(n) = n^2 - 21n$		$c(n)$ ist in $O(n^2)$ W oder F
$d(n) = n^3 + 3n$		$d(n)$ ist in $\Omega(n^2)$ W oder F

11. Bestimmen Sie die asymptotische Laufzeit des folgenden Algorithmus in der Θ-Notation. Begründen Sie Ihre Antwort kurz.

```
void f(int n) {
  int x = 0;
  for (int i = 1; i <= n; i *= 2) {
    for (int j = 0; j < n; j += i) {
      x++;
    }
  }
}
```

12. Bestimmen Sie die asymptotische Laufzeit des folgenden Algorithmus im Best Case und im Worst Case jeweils in der Θ-Notation. Geben Sie außerdem an, welche Eigenschaft das Eingabearray im jeweiligen Fall besitzt.

```
boolean containsPairWithSum(int[] arr, int x) {
 for (int i = 0; i < arr.length; i++) {
  for (int j = arr.length - 1; j > 0; j--) {
   if (arr[i] + arr[j] == x) {
    return true;
   }
  }
 }
 return false;
}
```

13. Bestimmen Sie die asymptotische Laufzeit des folgenden Algorithmus im Best Case und im Worst Case jeweils in der Θ-Notation. Geben Sie außerdem an, welche Eigenschaft das Eingabearray im jeweiligen Fall besitzt.

```
boolean containsPairWithSum(int[] arr, int x) {
  for (int i = 0; i < arr.length; i++) {
    for (int j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] == x) {
        return true;
      }
    }
  }
  return false;
}
```

Dieses Übungsblatt ist Teil der Lehrveranstaltung *„Algorithmik“*.